

- **Gün 1** -

CIPHER = SIFRE = SIFIR

Protokoller öğrenilecek

Kriptoloji = kriptografi + kriptanaliz

kriptografi = Güvenli sistem tasarlama bilimi (matematiksel zorluklar bulunmalı)

kriptanaliz = ()

Formal Modelleri öğren

Kriptoloji, bilgiyi güvenli kılma ve bu güvenliği test etme süreçlerini inceleyen ana bilim dalıdır. Bu denklemin iki temel bacağı Kriptografi (savunma/şifreleme) ve Kriptanaliz (saldırı/şifre kırma) bileşenlerinden oluşur. Kriptoloji (Şifre Bilimi): Gizli haberleşmenin, veri gizliliğinin ve bütünlüğünün genel teorisini kuran çatı alandır. "Şifreli bir sistem nasıl kurulur?" ve "Bu sistem ne kadar güvenlidir?" sorularına yanıt arar.

1. Kriptografi (Şifre Yazımı / Savunma) Anlaşılır durumdaki bir veriyi (Açık Metin / Plaintext), yetkisiz kişilerin okuyamayacağı karmaşık bir formata (Şifreli Metin / Ciphertext) dönüştürme sanatıdır. Bilgi güvenliğinin yapıcı tarafıdır. Amacı: Verinin gizliliğini (sadece yetkili okusun), bütünlüğünü (yolda değişmesin) ve inkar edilemezliğini (gönderen kimliğini kanıtlasın) sağlamaktır. Ne Yapar? Şifreleme algoritmaları (AES, RSA, ECC) hazırlar, dijital imzalar üretir ve hash fonksiyonları geliştirir. Benzetme: Dayanıklı, kurcalanamaz ve sadece doğru anahtarla açılan yüksek güvenli bir kasa veya kilit tasarlamak.

2. Kriptanaliz (Şifre Analizi / Şifre Kırma) Kriptografik sistemlerin zayıf yönlerini, matematiksel açıklarını veya anahtarlarını bulmayı hedefleyen inceleme ve kırma disiplini. Şifreli metinden yola çıkarak, anahtara sahip olmadan orijinal metni elde etmeye çalışır. Amacı: Tasarlanan şifreleme sistemlerinin gerçekten güvenli olup olmadığını test etmek, olası açıkları tespit etmek veya düşman/rakip iletişimini deşifre etmektir. Ne Yapar? Matematiksel modeller, frekans analizleri, kaba kuvvet (brute-force) saldırıları ve kuantum hesaplama yöntemleri kullanarak algoritmaları zorlar. Benzetme: Tasarlanan o kasanın kilidini maymancıkla, dinleme cihazlarıyla veya kilit mekanizmasındaki üretim hatasını bularak izinsiz açmaya çalışmak. Neden İkisi Bir Aradadır? Kriptografi ile Kriptanaliz arasında sürekli bir kovalamaca vardır. Güvenli bir şifre sistemi üretebilmek için bir kriptologun, sistemine nasıl saldırılabileceğini (Kriptanaliz) çok iyi bilmesi

gerekir. Kriptanalizciler yeni bir açık buldukça, kriptograflar daha güçlü algoritmalar geliştirir. Kriptanalizin en bilinen tarihsel örneklerinden biri olan Enigma şifresinin kırılması mantığını veya modern şifre kırma yöntemlerini detaylandırmamı ister misin?

şifrelemenin çok kolay deşifrelemenin çok zor olması hedeflenmeli

şifrelemenin maliyeti çok yüksek olmamalı zaman veya para

Doğruluk

bu bir güvenlik koşulu değildir kullanılabilir koşuldur.

şifreleme mesajları kısaltılamaz.

Dört Güvenlik Hedefi

Gizlilik, Bütünlük, Kimlik Doğrulama, İnkâr Edilemezlik

Sezar Şifresi

Neden önemli Şifrelemeyi bir aritmetik işleme çevirir. İlk kez “şifreleme = fonksiyon” fikri somutlaştırılmıştır.

Modüler aritmetik çok kritik işlemlere dönüşebilir incelenmesi anlaşılması gerekiyor. **Negatif mod almaya dikkat.**

Örnek

Sezar Anahtar Uzayı

Anahtar uzayının büyük olması zorluk derecesini arttıracığından dikkatle incelenmeli.

Asal sayı nedir neden kriptolojide önemlidir?

Vigenere Şifresi Nedir?

Fikir (Bellaso, 1553) : Tek bir kaydırma yerine bir kaydırma dizisi kullanma amacını taşır. Anahtar bir kelimedir ve mesaj boyunca tekrar eder. 300 yıl kırılmadı.

One Time Pad

Christoper Paar (Prof. Dr. Christof Paar): Kriptoloji alanında dünyaca tanınan Alman akademisyen ve yazardır. Jan Pelzl ile birlikte yazdığı **"Understanding Cryptography"** (Kriptografiyi Anlamak) kitabı, üniversitelerde ve kriptolojiye girişte en popüler kaynaklardan biridir.

Kırıldı Ne demek – Saldırı modelleri

Model	Saldırganın Gücü	Örnek
COA	yalnızca C görür	frekans analizi, sezar kaba kuvvet
KPA	bazı (m,c) çiftlerini bilir	enigma crib, dosya başlıkları
CPA	istediği m^* 'yi şifreletir	açık anahtarlı kriptografide her zaman geçerli
CCA	istediği c' 'yi deşifrelettirir	Bleichenbacher (1998), padding oracle

Modern standart : CCA güvenli** olmayan şema ciddiye almaz. Bu yüzden textbook RSA değil RSA-OAEP; ham CBC değil AES-GCM**

Kriptolojide ve bilgi güvenliğinde bir şifreleme sisteminin veya algoritmanın **"kırılmış" (broken)** kabul edilmesi, o sistemin **tasarlandığı güvenlik seviyesini ve sunduğu matematiksel garantileri kaybetmesi** demektir.

Halk arasında "kırıldı" ifadesi genellikle bir şifrenin tamamen çözülmesi veya sistemin darmadağın edilmesi gibi algılansa da, akademik ve teknik kriptanalizde tanım çok daha spesifik.

Saldırı modellerinde bir şifrenin kırılmış sayılması şu durumları ifade eder:

- **Teorik Güvenlik Sınırının Aşılması:** Bir algoritma, tasarlanırken hedeflenen karmaşıklıktan **daha az bir çabayla (daha az adım veya zamanla)** analiz edilebiliyorsa kırılmış sayılır. Örneğin; 128-bitlik bir şifreyi kaba kuvvetle (brute-force) denemek 2^{128} işlem gerektirir. Eğer bir araştırmacı matematiksel bir açık bularak bunu 2^{110} işleme düşürürse, bu işlem pratikte yıllar alsa bile **algoritma teorik olarak kırılmıştır**.
- **Kaba Kuvvet (Brute-Force) Dışında Yol Bulunması:** Şifreleme anahtarını bulmanın tek yolu evrendeki tüm olasılıkları tek tek denemek olmalıdır. Eğer bu deneme yolunu baypas eden matematiksel bir kısayol (kriptanaliz yöntemi) bulunursa sistem kırılmış kabul edilir.
- **Tasarım Amacının İhlal Edilmesi:**
 - **Hash Algoritmalarında (örn. MD5, SHA-1):** Aynı çıktıyı veren iki farklı veri bulunduğunda (Çakışma / Collision), o hash algoritması kırılmış sayılır.
 - **Asimetrik Şifrelemede (örn. RSA):** Açık anahtardan gizli anahtarı elde etmeyi sağlayan matematiksel bir yöntem bulunduğunda sistem kırılır.

Saldırı Modellerine Göre "Kırılma" Türleri

Kriptanalizde bir şifreleme sistemine erişim seviyenize göre saldırı modelleri belirlenir. Şifrenin ne seviyede "kırıldığı" bu modellere göre değişir:

AÇIK ANAHTAR ŞİFRELEME

RSA

RSA şifreleme algoritması, 1970'lerin ortalarında bilgisayar ağlarının gelişmesiyle ortaya çıkan "**Güvenli Anahtar Değişimi**" krizine bir çözüm olarak doğdu.

RSA'dan önce yalnızca **Simetrik Şifreleme** biliniyordu. Bu sistemde mesajı şifreleyen anahtar ile çözen anahtar aynıydı. Dolayısıyla iki kişinin gizlice haberleşebilmesi için önce bu gizli anahtarı birbirine güvenli bir şekilde (örneğin fiziki bir kuryeyle) ulaştırması gerekiyordu. İnternetin ve küresel ağların emekleme aşamasında olduğu o dönemde, milyonlarca kullanıcının birbirine güvenli anahtar ulaştırması imkansız bir problemdi.

Kronolojik Gelişim ve Ortaya Çıkış Süreci

Diffie-Hellman Kavramı

1976

Whitfield Diffie ve Martin Hellman, "**Açık Anahtarlı Kriptografi**" (**Public-Key Cryptography**) konseptini teorik olarak ortaya attı. Fikir şuydu: Herkesin bir **Açık Anahtarı (Public Key)** ve bir **Gizli Anahtarı (Private Key)** olmalıydı. Biri size mesaj atacaksa herkesin gördüğü açık anahtarınızı kullanmalı, şifreyi ise sadece sizdeki gizli anahtar çözebilmeliydi. Ancak bunu sağlayacak matematiksel bir fonksiyon henüz bulunamamıştı.

MIT'de Araştırma Grubu

1977

MIT'de (Massachusetts Institute of Technology) çalışan üç araştırmacı — **Ron Rivest** (bilgisayar bilimci), **Adi Shamir** (bilgisayar bilimci) ve **Leonard Adleman** (matematikçi) — Diffie-Hellman'ın teorisini gerçeğe dönüştürmek için çalışmaya başladı. Rivest ve Shamir sürekli yeni matematiksel fonksiyonlar tasarlıyor, Adleman ise bir matematikçi olarak bu sistemlerin açıklarını bulup kırıyordu.

Tek Yönlü Matematiksel Fonksiyonun Bulunuşu

1977 Gece Yarısı

Nisan 1977'de, 40'tan fazla başarısız denemenin ardından Ron Rivest bir gece uyuyamadı ve asal sayıların çarpımına dayanan matematiksel formülü geliştirdi. Adleman sistemi inceledi ve bu kez kırmayı başaramadı. Algoritma, mucitlerinin soyisimlerinin baş harfleriyle **RSA** (Rivest-Shamir-Adleman) olarak adlandırıldı.

Patent ve Yayınlama

1977 - 1983

Algoritma 1977'de *Scientific American* dergisinde halka açıklandı ve çözülmesi için \$125\$ basamaklı bir şifreli metin (RSA-129) sunuldu (bu şifre ancak 1994 yılında kırılıldı). MIT, 1983 yılında RSA için ABD patenti aldı.

Kilit Nokta: Şifrenin Arkasındaki Matematiksel Mantık

RSA'nın başarısı, asal sayıların matematiksel bir özelliğine dayanır: "**Çarpmak çok kolay, çarpanlara ayırmak çok zordur.**"

- İki adet devasa asal sayıyı (\$p\$ ve \$q\$) birbiriyle çarparak devasa bir \$N\$ sayısı elde etmek bilgisayarlar için saniyenin binde biri kadar sürer.

- Ancak sadece bu N sayısını bilerek, onun hangi iki asal sayıdan ($p \times q$) oluştuğunu bulmak (çarpanlara ayırma / factorization) günümüzdeki en güçlü süper bilgisayarlar için bile binlerce yıl sürer.

RSA, bu "kolay hesaplanan ama geri döndürülmesi imkansız" matematiksel tuzağı (Trapdoor One-Way Function) kullanarak dijital dünyanın en temel güvenlik altyapısını oluşturdu.

Tarihsel Not (GCHQ ve Clifford Cocks):

Britanya İstihbarat Teşkilatı (GCHQ) çalışanı **Clifford Cocks**, RSA'dan 4 yıl önce (1973'te) aynı matematiksel modeli gizli bir askeri proje olarak keşfetmişti. Ancak bu çalışma gizli devlet sırrı sayıldığı için 1997 yılına kadar kamuoyuna açıklanmadı. Bu yüzden buluşun resmi tarihi ve patenti Rivest, Shamir ve Adleman'a aittir.

Mod Kolaylık ;

$\frac{1}{2} = (\text{mod} 5)$ için $1 + 5/2 = \text{mod } 5$ yani 1'e 5 ekleyerek bolumun kalanını bulmak kolaylaşır

Hızlı Üs Alma ;

$3^{13} = x (\text{mod} 17)$ için üssün 13 ün iki ye bölünerek binaryi kodu alınır 1101 olur ve sağdan basamakların 2 uzeri değerleri toplanır yani $2^0 + 2^1 + 2^2 + 2^3 = 13$

Ardından $(\text{mod } 17)$ ye göre işlemleri kolay yapabilmek için $3^1 (\text{mod} 17) \cdot 3^4 (\text{mod} 17) \cdot 3^8 (\text{mod} 17)$ sonucuna gidilir.

Bir sayının **asal sayı** olup olmadığını (yani sadece 1 'e ve kendisine bölünüp bölünmediğini) tespit etmek için temel matematik mantığından başlayıp optimize edilen algoritmalar kullanılır.

1. Temel Deneme Bölmesi (Trial Division) Algoritması

Bir N sayısının asal olup olmadığını anlamanın en ilkel yolu, 2 'den $N-1$ 'e kadar olan tüm sayılara bölmeyi denemektir.

- Sorun:** Sayı büyüdükçe denenecek sayı miktarı devasa boyuta ulaşır. Örneğin 100 basamaklı bir sayı için bu yöntem bilgisayarla bile imkansızdır.

2. Optimize Edilmiş Algoritma ($\sqrt{n}$ Mantığı)

Bir sayının çarpanları her zaman çiftler halinde bulunur. Örneğin 36 sayısı için:

- 1×36
- 2×18
- 3×12

- **6×6** (Karekök sınırı: $\sqrt{36} = 6$)
- 9×4
- $12 \times 3 \dots$

Karekökten ($\sqrt{n}$) sonraki çarpanlar, önceki çarpanların yer değiştirmiş halidir. Bu yüzden **bir sayıyı $\sqrt{n}$ 'e kadar test etmek yeterlidir.**

Mantıksal Adımlar (Pseudo-Code):

1. $n \leq 1$ ise **Asal Değildir.**
2. $n = 2$ veya $n = 3$ ise **Asaldır.**
3. n çift sayı ise (ve 2'den büyükse) **Asal Değildir.**
4. $i = 3$ 'ten başlayarak, $i \times i \leq n$ olduğu sürece i 'yi 2 'şer artırarak ($3, 5, 7, 9 \dots$) $n \bmod i == 0$ kontrolü yap:
 - Eğer kalan 0 ise **Asal Değildir.**
5. Hiçbirine bölünmediyse **Asaldır.**

3. Kriptografide Kullanılan Devasa Asal Sayı Testleri

RSA gibi kriptografik sistemlerde yüzlerce basamaklı asal sayılar kullanılır. Bu büyüklükteki sayılar için $\sqrt{n}$ yöntemi bile çok yavaş kalır. Bu yüzden **Olasılıksal Asallık Testleri** kullanılır:

- **Miller-Rabin ve Fermat Asallık Testleri:** Bir sayının %100 değil, örneğin %99.9999999999999999% ihtimalle asal olduğunu milisaniyeler içinde kanıtlar.
- **AKS Asallık Testi:** Bir sayının kesin olarak asal olup olmadığını deterministik (kesin) ve polinomik zamanda çözen ilk algoritmadır.

C# Kod Örneği (Optimize Yöntem)

C#

```
public bool IsPrime(long number)
{
    if (number <= 1) return false;
    if (number == 2) return true;
    if (number % 2 == 0) return false;
    long boundary = (long)Math.Floor(Math.Sqrt(number));
    for (long i = 3; i <= boundary; i += 2)
    {
        if (number % i == 0)
            return false;
    }
    return true;
}
```


RSA ve Aralarında ASALLIK

İki sayının **aralarında asal** olması, \$1\$'den başka ortak pozitif bölenlerinin olmaması demektir.

Bir \$A\$ sayısı ile \$9\$ sayısının aralarında asal olup olmadığını bulmak için tek bir kural vardır: **A sayısının \$3\$'e bölünmemesi gerekir.**

Mantığı ve Adım Adım Hesaplanışı

1. 9 Sayısının Asal Çarpanlarına Bakılır:

- \$9 = 3^2\$
- Yani 9 sayısının tek asal böleni **\$3\$**'tür.

2. Kural Belirlenir:

- Bir sayının 9 ile ortak bir çarpanının (1 hariç) olabilmesi için o sayının da 3'e bölünmesi gerekir
- Dolayısıyla **\$3\$'ün katı olmayan her pozitif tam sayı 9 ile aralarında asaldır.**

1 ile 9 Arasındaki Sayılar İçin Kontrol (Örnek)

Sayı	3'e Bölünür mü?	9 ile Aralarında Asal mı?	Nedeni
1	Hayır	Evet	1 tüm pozitif tam sayılarla aralarında asaldır.
2	Hayır	Evet	Ortak bölenleri sadece 1.
3	Evet	Hayır	Ortak bölenleri 3'tür.

Sayı	3'e Bölünür mü?	9 ile Aralarında Asal mı?	Nedeni
4	Hayır	Evet	Ortak bölenleri sadece 1.
5	Hayır	Evet	Ortak bölenleri sadece 1.
6	Evet	Hayır	Ortak bölenleri 3'tür ($\text{EBOB}(6,9)=3$).
7	Hayır	Evet	Ortak bölenleri sadece 1.
8	Hayır	Evet	Ortak bölenleri sadece 1.

Pratik Formül: Euler Totient (ϕ) Fonksiyonu

Eğer "9'dan küçük kaç tane sayı 9 ile aralarında asaldır?" sorusunun cevabını direkt bulmak istersen **Euler Totient (ϕ)** formülü kullanılır:

$$\phi(n) = n \times \left(1 - \frac{1}{p}\right)$$

Burada p , sayının asal çarpanıdır (9 için $p=3$):

$$\phi(9) = 9 \times \left(1 - \frac{1}{3}\right) = 9 \times \frac{2}{3} = 6$$

Yani 9'dan küçük **6 tane** sayı (1, 2, 4, 5, 7, 8) 9 ile aralarında asaldır. Bu matematiksel fonksiyon RSA şifrelemesinin anahtar üretme adımıda doğrudan kullanılan temel yapıdır.

Alice

1. İki büyük asal seç p, q
2. $N = p \cdot q$
3. $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$
4. $\text{Obeb}(e, \phi(N)) = 1$ olacak şekilde e seç
5. $d = e^{-1} \pmod{\phi(N)}$

AÇIK : n, e

GİZLİ : $d, \phi(N)$

5 inci adımı ayrıca incelemek gerekir

Bob

m = mesajı var ve $c = m^e \pmod{N}$

Alice c mesajını aldığı anda $c^d = (m^e)d = m^{ed} \pmod{N} = m^{ed} = m \pmod{N}$

Kitap Önerileri Dan Boneh ve Vitor Shoup, information theory araştır

Diffie Helman Algoritmasının zorluğu neredeydi

Diffie-Hellman algoritmasının güvenliği ve matematiksel zorluğu, **Ayrık Logaritma Problemi**'ne (Discrete Logarithm Problem) dayanır.

Tıpkı RSA'daki asal çarpanlara ayırma gibi, Diffie-Hellman'da da bir yönde işlem yapmak bilgisayarlar için çok kolayken, tersine işlem yapmak imkansıza yakındır.

Matematiksel Zorluğun Mantığı

Algoritma modüler aritmetik (kalanlı bölme) ve üslü sayılar üzerine kuruludur:

$$y = g^x \pmod{p}$$

Burada:

- g : Ortak taban (küçük bir tam sayı)
- p : Devasa bir asal sayı (örneğin 2048-bit / yüzlerce basamaklı)

- x : Tarafın seçtiği **gizli anahtar**
- y : Dışarıya yayınlanan **açık anahtar**

1. Kolay Yön (Şifreleme / Anahtar Üretimi)

Sana g , x ve p verilirse; $g^x \bmod p$ hesabını yapmak bilgisayar için milisaniyeler sürer. Buna **Modüler Üs Alma** denir.

2. İmkansız/Zor Yön (Saldırı / Deşifre Etme)

Ağı dinleyen bir saldırgan (Eve); herkese açık olan g , p ve hesaplanan y değerlerini görür. Saldırganın gizli anahtar olan x 'i bulması için şu işlemi yapması gerekir:

$$x = \log_g(y) \bmod p$$

İşte bu işleme **Ayrık Logaritma** denir. Normal matematikte logaritma almak kolayken, devasa bir p asal sayısı modunda çalışıldığında bu işlemi tersten çözecek **hızlı bir matematiksel kısayol veya formül yoktur**.

Saldırgan Neden Çözemez?

Saldırganın x 'i bulmak için deneyebileceği yöntemler ve karşılaştığı engeller şunlardır:

- **Kaba Kuvvet (Brute-Force):** x yerine $1, 2, 3 \dots$ koyarak y 'yi elde etmeye çalışmak. p sayısı 2048-bit seçildiğinde olası x sayısı evrendeki atom sayısından daha fazladır. Tüm dünya süper bilgisayarları toplansa bu deneme evrenin yaşından uzun sürer.
- **Matematiksel Kısayol Yokluğu:** Modüler aritmetikte sonuçlar bir doğru üzerinde düzenli artmaz; 1 ile $p-1$ arasında rastgele sıçar. Bu yüzden grafikte bir yön veya eğilim izleyerek cevaba yaklaşmak mümkün değildir.

Günümüzdeki Durumu ve Eliptik Eğriler (ECDH)

Ayrık logaritma problemini çözmek için geliştirilen en gelişmiş klasik algoritma **General Number Field Sieve (GNFS)**'dir. Bu algoritma nedeniyle Diffie-Hellman'ın güvenli kalabilmesi için p asal sayısının en az **2048-bit veya 3072-bit** gibi devasa boyutlarda seçilmesi gerekir. Günümüzde bu işlemi daha küçük anahtar boyutlarıyla çok daha zor hale getirmek için klasik modüler aritmetik yerine **Eliptik Eğriler (ECDH - Elliptic Curve Diffie-Hellman)** kullanılır. Eliptik eğrilerdeki ayrık logaritma problemini çözmek klasik Diffie-Hellman'a göre katbekat daha zordur.

- **Gün 3 24.08.2026 –**

Sinyal ve gürültü oranı nedir?

- **Gün 4 25.08.2026 –**

- **Gün 5 26.08.2026 –**

- **Gün 6 27.08.2026 –**

Soru : blockchain de iş kanıtı neden gerekli ve anlama geliyor metaforla anlatırmısın

İş Kanıtı (Proof of Work - PoW), başında bir patron, banka veya devlet bulunmayan **merkeziyetsiz** bir sistemde, herkesin aynı gerçeğe ve veri defterine güvenmesini sağlayan **güvenlik ve uzlaş (konsensüs) mekanizmasıdır**.

Sistemde bir yönetici olmadığı için "Bu işlemleri kim doğru sırayla yazacak?" ve "Ağdaki kötü niyetli kişilerin sahte işlemler eklemesini nasıl engelleyeceğiz?" sorularının cevabını İş Kanıtı verir.

İş Kanıtı'nı Anlamak İçin Metafor: "Dev Piyango ve Şifreli Çelik Kasa"

Hayal edin ki dünyadaki binlerce insan aynı açık hesap defterine (blokzincire) finansal işlemler yazmak istiyor. Ama aralarında hiçbir güven ilişkisi yok.

1. **Sayfa Doldurma (Blok Oluşturma):** Son 10 dakikada yapılan tüm para transferleri bir A4 kağıdına yazılıyor.
2. **Kilitli Kasa ve Şifre (Zor Matematiksel Problem):** Bu sayfayı deftere ekleyip ödülü kapmak isteyen herkes (madenciler), bu sayfayı şifreli bir çelik kasanın içine koymak zorunda. Kasanın şifresini bulmanın hiçbir mantıksal formülü veya kısayolu yoktur; **tek yol rakamları milyarlarca kez denemektir** (Örn: 0001, 0002... 89432...).

3. **Maddi Emek (İş / Work):** Kasayı açacak doğru şifreyi bulmak için madencilerin bilgisayarları devasa miktarda elektrik harcar ve işlem gücü tüketir. Bu harcanan gerçek dünya kaynağı, yapılan "İş"tir.
4. **İspat (Proof):** İçinden biri şifreyi bulduğu an "*Buldum!*" diye bağırır. Diğer herkes o şifreyi kendi kasasında sadece 1 saniyede deneyip "*Evet, gerçekten kilit açıldı, doğru şifre bu*" der. Doğru şifrenin sunulması, o kişinin arkasında saatlerce elektrik ve emek harcadığının "Kanıtı"dır.

İş Kanıtı Neden Şarttır? (3 Temel Sebebe)

1. Hile Yapmayı "Aşırı Pahalı" Hale Getirmek

Eğer yeni bir sayfa (blok) eklemek için hiçbir emek veya maliyet gerekmeseydi, kötü niyetli biri saniyeler içinde binlerce sahte işlem sayfası üretebilir ve "Benim defterim gerçek" diyebilirdi.

İş Kanıtı sayesinde geçmişteki bir sayfayı değiştirmek isteyen saldırgan, **o günden bugüne kadar harcanan tüm elektriği ve matematiksel emeği tek başına tekrar harcamak zorundadır**. Bu da trilyonlarca dolarlık elektrik ve donanım maliyeti demektir. Yani sistemi hacklemek, dürüst kalıp madencilik yapmaktan çok daha pahalıya gelir.

2. Çifte Harcamayı (Double Spending) Önlemek

Aynı parayı iki kişiye birden harcama sahtekarlığını engeller. Sayfalar (bloklar) zamansal olarak zorlu bir yarışla birbirine kilitlendiği için, hangi işlemin önce yapıldığı kesinleşmiş olur.

3. Merkeziyetsiz Lider Seçimi (Rastgelelik)

Sıradaki bloğu kimin yazacağını seçen merkezi bir hakem yoktur. İş Kanıtı algoritması, işlem gücüne bağlı olarak **tamamen adil ve piyango mantığıyla** bir sonraki bloğu yazacak kişiyi (madenci) belirler.

Özeti: Asimetrik Mantık

İş Kanıtı'nın en önemli özelliği **Bulması Çok Zor, Kontrol Etmesi Çok Kolay** olmasıdır.

Örnek: Bir Rubik Küpü'nü karıştırıp sıfırdan çözmek uzun zaman ve emek gerektirir (Çözüm Aşaması / İş). Ancak birisi çözülmüş küpü önünüze koyduğunda, onun gerçekten çözümlü çözülmeyeceğini anlamak 1 saniyenizi alır (Kontrol Aşaması / Kanıt).

PoW; dijital dünyada güvenin arkasına **fiziksel dünyadaki enerji ve sermaye maliyetini** koyarak, hile yapmayı imkansız kılan sistemdir.

Soru : bugunki blockchain yapısı ile ilk versiyon arasında en önemli farklar nelerdir bugunki yapı kacinci versiyondadır bu versiyonların piyasaya surulmesini kim kontrol eder bir otorite var mı

İlk sürümlerden günümüze blokzincir (Blockchain) teknolojisi tek bir kurum veya şirket tarafından değil; **açık kaynak kodlu geliştirici toplulukları, araştırmacılar ve ağ katılımcılarının ortak mutabakatı (consensus)** ile evrilmiştir.

Sistemin kat ettiği gelişim "**Blokzincir Sürümleri/Nesilleri**" (**Generations**) olarak adlandırılır.

1. Blokzincir Nesilleri (Versiyonlar)

Kripto dünyasında versiyonlar resmi bir yazılım güncellemesi gibi tek bir merkezden duyurulmaz; teknolojinin yeteneklerine göre dönemsel nesillere ayrılır:

Blokzincir 1.0 — Kripto Paralar (Örn: Bitcoin / 2009)

- **Amaç:** Yalnızca kişiden kişiye (P2P) güvenli ve merkeziyetsiz dijital para transferi.
- **Karakteristik:** Akıllı sözleşme yok, işlem kapasitesi düşük (~7 TPS), İş Kanıtı (Proof of Work) mantığı.

Blokzincir 2.0 — Akıllı Sözleşmeler ve DApp'ler (Örn: Ethereum / 2015)

- **Amaç:** Sadece para transferi değil, ağ üzerinde programlanabilir kodlar (Smart Contracts) çalıştırmak.
- **Karakteristik:** Merkeziyetsiz uygulamalar (DApps), Finansal Araçlar (DeFi), Ethereum Sanal Makinesi (EVM).

Blokzincir 3.0 — Ölçeklenebilirlik ve Birlikte Çalışabilirlik (Örn: Cardano, Polkadot, Solana / 2018+)

- **Amaç:** Yüksek işlem ücretleri ve yavaşlık sorununu çözmek.
- **Karakteristik:** Proof of Stake (PoS) geçişi, saniyede binlerce işlem (TPS), farklı zincirlerin birbiriyle konuşabilmesi (Cross-chain).

Blokzincir 4.0 — Modüler Yapı, Katman-2 ve Kurumsal Entegrasyon (Günümüz Mimarisi)

- **Amaç:** Blokzinciri arka planda görünmez kılarak internet ölçeğinde kullanıma açmak.
- **Karakteristik: ZK-Rollup'lar, Modüler Yapılar** (Celestia vb.), Yapay Zeka (AI) ve Gerçek Dünya Varlıklarının (RWA - Real World Assets) zincire aktarılması.

2. İlk Versiyon ile Bugün Arasındaki En Önemli Farklar

- **Fikir Birliği Mekanizması:** İlk versiyonlarda devasa elektrik ve donanım tüketen **Proof of Work (PoW)** kullanılırken; günümüzde doğrulayıcıların varlık kilitlediği çevreci ve hızlı **Proof of Stake (PoS)** veya **Proof of History (PoH)** kullanılır.
- **Mimari Değişim (Monolitik'ten Modüler'e):** İlk versiyonlarda her düğüm (node) veriyi saklar, doğrular ve işlerdi. Bugünkü yapıda işlemler **Katman-2 (Layer-2)** protokollerinde hızlıca hesaplanıp, ana zincire (Layer-1) sadece güvenlik ve doğrulama özeti yazılır.
- **Sanal Makineler ve Esneklik:** Sadece sabit cüzdan bakiyesi tutan sistemlerden; karmaşık finansal işlemleri, NFT'leri ve oyunları çalıştıran gelişmiş Sanal Makinelere (EVM, WASM, Move VM) geçilmiştir.

3. Bu Versiyonları ve Piyasaya Sürülmesini Kim Kontrol Eder?

Bu süreçleri kontrol eden merkezi bir otorite, devlet veya tek bir şirket YOKTUR.

Açık kaynaklı ve kamuya açık (public) blokzincirlerde güncellemeler ve versiyon geçişleri **Merkeziyetsiz Yönetişim (Decentralized Governance)** ile ilerler:

1. **Geliştirme Teklifleri (EIP/BIP):** Geliştiriciler veya topluluk üyeleri sisteme bir güncelleme getirmek istediğinde bunu bir geliştirme teklifi (Örn: Ethereum için *EIP - Ethereum Improvement Proposal*) olarak yayınlar.
2. **Topluluk ve Madenci/Doğrulayıcı Oylaması:** Ağdaki Düğümler (Nodes), Madenciler/Doğrulayıcılar (Validators) ve Borsa/Yazılım geliştiricileri bu koda onay verip vermeyeceklerini tartışır.
3. **Hard Fork / Soft Fork (Yol Ayrımı):**

- **Çoğunluk Kabul Ederse:** Ağ katılımcılarının büyük kısmı yeni yazılım sürümünü bilgisayarlarına indirip çalıştırır ve sistem güncellenmiş olur.
- **Anlaşmazlık Olursa:** Eğer topluluk ikiye bölünürse zincir çatallanır (Hard Fork) ve ortaya iki farklı kripto para çıkar. (Örn: 2017'de blok boyutu tartışması yüzünden Bitcoin'den Bitcoin Cash'in doğması veya 2016'da Ethereum ve Ethereum Classic'in ayrılması).

Kısacası; sistemin kuralını veya kodunu yazan yazılımcılar olsa dahi, o kodu çalıştırıp çalıştırmama kararı dünya genelindeki binlerce bağımsız **sunucu sahibine (düğümlere)** aittir.

Günümüz katmanlı mimarisindeki ZK-Rollup (Zero-Knowledge) teknolojisinin detaylarını veya Ethereum'un PoS geçişindeki (The Merge) yönetim sürecini incelemek ister misin?

Soru : peki aynı anda bir çok madenci neden sürekli bitcoin üretmek için elektrik harcamaya devam ediyor aralarından sadece bir kişi doğru şifreyi bulması demek aslında bir çoğunun gereksiz kaynak ve enerji tüketmesi demek değil mi?

Evet, tespiti tam olarak doğru bir noktaya değiniyor.

Yazılım ve sistem mühendisliği perspektifinden bakıldığında İş Kanıtı (Proof of Work), **devasa bir kaynak israfı ve verimsizlik** gibi görünür. Dünya genelinde yüz binlerce madenci cihaz aynı bulmacayı çözmek için çalışır, ancak **sadece ilk bulan tek bir kişi** ödülü (6.25 BTC/3.125 BTC + işlem ücretleri) kazanır. Geriye kalan tüm madencilerin harcadığı elektrik, matematiksel açıdan o an için tamamen boşa gitmiş olur.

Buna rağmen sistemin bu şekilde çalışmaya devam etmesinin arkasındaki nedensellik ve ekonomi mantığı şu şekildedir:

1. İsrar Gibi Görünen Şey Aslında "Güvenlik Maliyetidir"

Geleneksel dünyada bir bankanın güvende kalması için ne harcanır?

- Zırhlı araçlar, güvenlik görevlileri, devasa çelik kasalar, kameralar, yüksek binalar ve bunların elektrik giderleri...

Bitcoin ağında fiziki binalar veya ordu yoktur. **O binaların ve güvenlik sistemlerinin yerini madencilerin harcadığı elektrik alır.**

Ağdaki madenci sayısı ne kadar çok olursa ve toplam harcanan elektrik (Hash Rate) ne kadar büyürse, dışarıdan kötü niyetli bir saldırganın ağı manipüle etme maliyeti o kadar imkansız hale gelir. Ağdaki "israf" denilen o devasa enerji, **sistemin hacklenemezliğini sağlayan kırılmaz bir güvenlik kalkanıdır.**

2. Oyun Teorisi ve Ekonomik Teşvik (Game Theory)

Madenciler bu enerjiyi hayır kurumu oldukları için değil, **istatistiksel bir piyango beklentisi** yüzünden harcarlar.

- Bir madenci tek başına doğru şifreyi bulma şansının düşük olduğunu bilir. Bu yüzden günümüzde madenciler güçlerini **Madencilik Havuzlarında (Mining Pools)** birleştirirler.
- Havuzdaki binlerce cihazdan **herhangi biri** şifreyi bulduğunda, kazanılan ödül havuza katkı sağlayan tüm madencilere harcadıkları elektrik ve işlem gücü oranında dağıtılır.
- Yani madenci her blokta piyangoyu tek başına kazanmasa bile, havuza sağladığı katkı kadar **düzenli bir gelir** elde eder. Bu da harcadığı elektriğin karşılığını ticari olarak almasını sağlar.

3. "Tamamen Gereksiz" mi? (Zorluk Ayarlaması - Difficulty Adjustment)

Sistemde madenci sayısı arttıkça şifreler zorlaşır, madenci sayısı azaldıkça şifreler kolaylaşır. Ağ, ortalama **10 dakikada bir** 1 bloğun bulunmasını sağlayacak şekilde zorluğu otomatik ayarlar.

Eğer madencilerin büyük kısmı "Çok elektrik gidiyor, ben çıkıyorum" derse:

1. Ağın toplam işlem gücü düşer.
2. Zorluk derecesi otomatik olarak aşağı çekilir.
3. Bulmaca kolaylaştığı için daha az elektrikle blok bulunmaya başlar.

Yani harcanan enerji miktarı sabit bir kural değil, **Bitcoin'in piyasa değeri ile elektriğin maliyetinin serbest piyasada dengeye ulaştığı dinamik bir noktadır.**

Özetle

Sistemdeki **"boşa harcanan iş" (Wasted Work)** bir tasarım hatası değil, sistemin bilerek istenen en temel özelliğidir.

Saldırganlara *"Bu ağı manipüle etmek istiyorsan, dünyadaki tüm bu madencilerin toplamından daha fazla elektriği göze almalısın"* mesajı verilir. Tam olarak bu verimsizlik yüzünden günümüzde **Proof of Stake (PoS / Pay Kanıtı)** gibi elektrik harcamayan alternatif sistemler (örneğin Ethereum 2.0) geliştirilmiştir. PoS sistemlerinde madencilik ve elektrik harcaması yoktur; bunun yerine sisteme varlık/para kitleme (Staking) mantığı kullanılır.

Soru : peki nonce blogu ne işe yarar ne amaçla kullanılır

Nonce (*Number Used Once - Sadece Bir Kez Kullanılan Sayı*), madencilerin İş Kanıtı (Proof of Work) bulmacasını çözmek için **sürekli değiştirdikleri tek değişkendir (değerdir)**.

Ne İş Yapar ve Neden Gerekli?

Blok başlığı (Block Header) içinde yer alan diğer tüm veriler **sabit veya sabite yakındır**:

- **Önceki Bloğun Hash'i**: Değiştirilemez (geçmişe bağlıdır).
- **İşlemler (Merkle Root)**: Değiştirilemez (o anki transferlerdir).
- **Zaman Damgası (Timestamp)**: O anki saattir, keyfi değiştirilemez.

Kriptografik hash fonksiyonlarının (SHA-256) mantığı gereği: **Aynı girdi, her zaman aynı çıktıyı verir.**

Eğer madencinin elindeki tüm veriler sabit olsaydı, bunları hash'lediğinde **her defasında aynı sonucu** elde ederdi. Ağın istediği *"baştaki sıfırlarla başlayan"* özel hash değerini yakalamak için girdide en az 1 bitlik bir değişkenin değişmesi gerekir.

İşte **Nonce**, madencinin girdiyi değiştirebilmesi için bloğun içine koyulmuş **4 baytlık (32-bit) boş bir sayaç alanıdır**.

Çalışma Mantığı (Sayısal Örnek)

Ağın hedefi: "Sonucu en az 4 tane sıfır ile başlayan bir hash bul" olsun.

1. **Deneme 1:** [Sabit Veriler] + [Nonce: 0] $\rightarrow$ Hash: 8a7f... (Başarısız)
2. **Deneme 2:** [Sabit Veriler] + [Nonce: 1] $\rightarrow$ Hash: 3f1b... (Başarısız)
3. **Deneme 3:** [Sabit Veriler] + [Nonce: 2] $\rightarrow$ Hash: 99e4... (Başarısız)

...

4. **Deneme 485.102:** [Sabit Veriler] + [Nonce: 485102] $\rightarrow$ Hash: 0000a2f8... (**BAŞARILI!**)

Madenci aranan hedef koşulu sağlayan 485102 değerini bulduğu an, bu **Nonce** değerini bloğun içine yazar ve ağa yayınlar.

Diğer Madenciler Ne Yapar?

Diğer madenciler \$485.102\$ defa deneme yapmaz. Bloğu aldıklarında sadece Nonce = 485102 değerini sabit verilerle birleştirip **tek bir kez** SHA-256 işleminden geçirirler. Output 0000... ile başlıyorsa bloğu anında onaylarlar.

Özetle Nonce; madencilerin hedef hash'i bulana kadar **üzerinde sayı saydığı bir sayaç/çarktır**.

- Oğuz YAYLA makalesini inceleyin, Adli Emanet nedir?
- Kriptopara Adli Emaneti
-
-

- Doğal Nükleer Reaktör Nedir 5dk sunum hazırlanacak
- Engür hocanın “teleffuz edilebilir parolalar” makalesini google da arat ve incele
- 100 kez hello world yaz ama For do while while go to elle tek tek copy paste recursive kullanmak yok (dikkatlice istişare edilerek yapılacak)